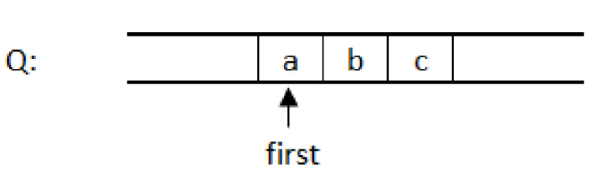
6. Gyakorlat

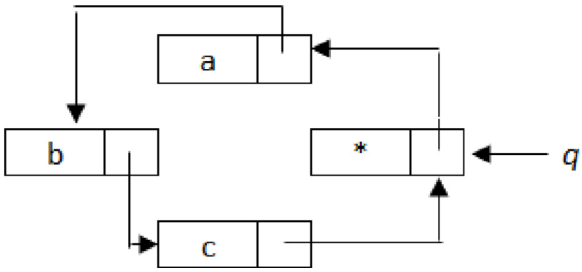
Láncolt listák, Sor

Sor megvalósítása ciklikus listával

- egyirányú ciklikus lista ábrázolja a sort
  - joker elem a sor végén, az első elemre mutat
- üres lista: csak a joker elemet tartalmazza, next-je önmagára mutat
- lista kezelése: egy pointerrel

Sor típusa

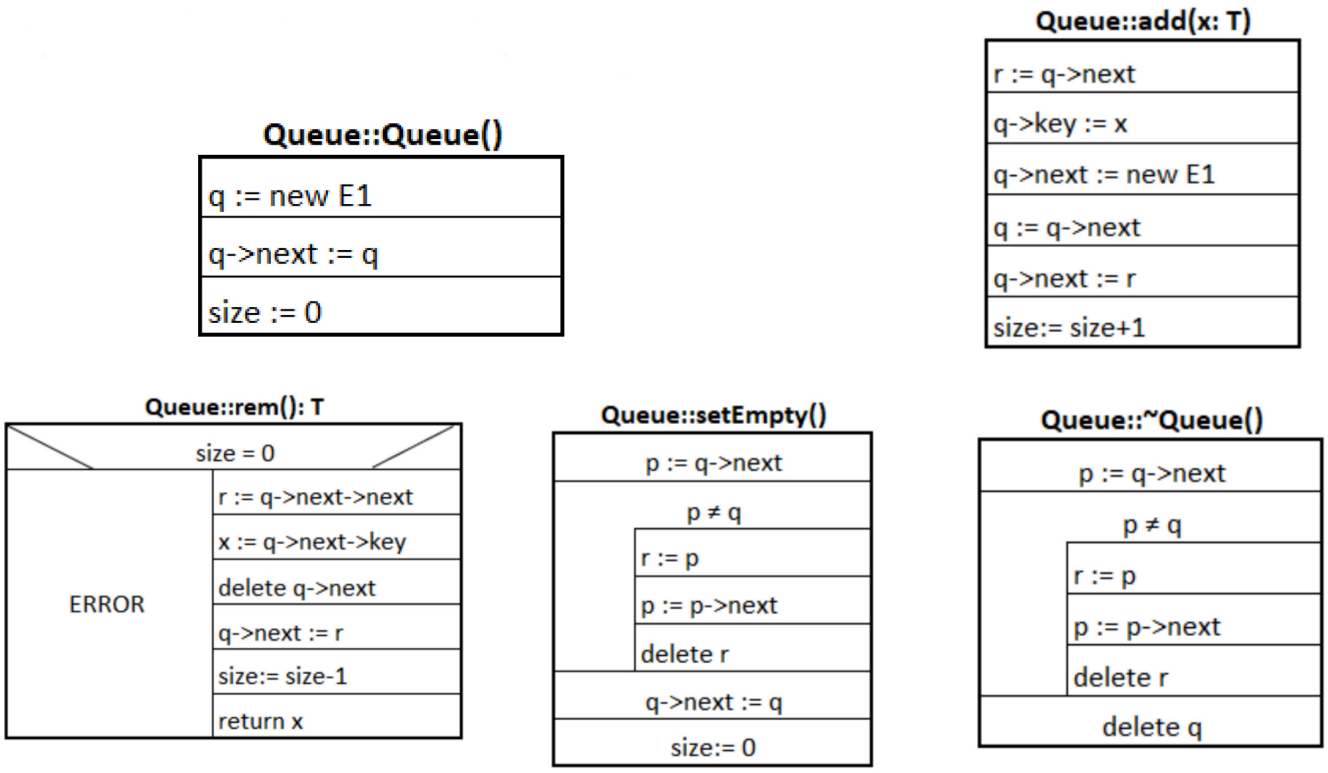




Queue
-q: El* *//a joker elemre mutató pointer* -size: N
+ Queue() + add(x: T) *// új elem hozzáadása a sor végére* + rem(): T *// a sor elején lévő elem eltávolítása* + first() : T *// a sor elején lévő elem lekérdezése* + length(): N + isEmpty(): B + ~Queue() + setEmpty()

- `rem()`
  - az első elem azonnal elérhető q segítségével
- `add()`
  - az új érték a joker elembe kerül
  - új joker elemet hozunk létre
  - q-val az új joker elemre mutatunk

Műveletek



S1L szétválogatása

L legyen egy egész számokat tartalmazó S1L lista, a kulcsok egész számok, a lista növekvően rendezett.

Fűzzük szét az elemeket L1 és L2 S1L listákba. L1-be a páros, L2-be a páratlan kulcsú elemeket kell fűzni, L1 és L2 is legyen rendezett! (L lehet üres, lehet, hogy csupa páros, vagy csupa páratlan elemből áll.)

Például legyen L kezdetben a következő lista:

L → 1 → 2 → 3 → 4 → 6 → 7 → 12

Ekkor a végeredmény:

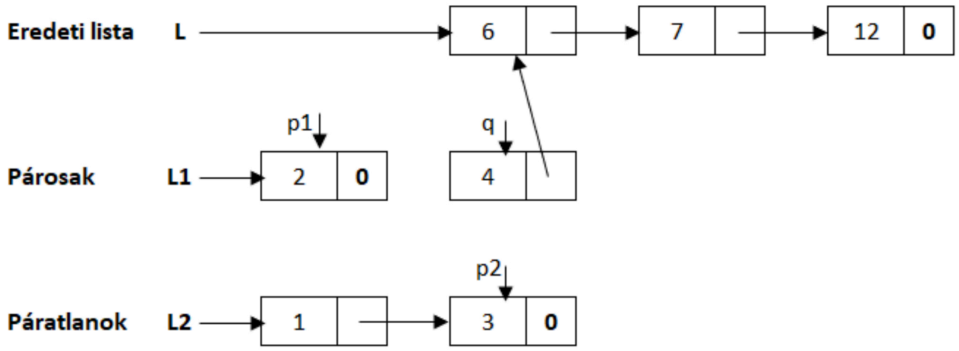
L1 → 2 → 4 → 6 → 12 L2 → 1 → 3 → 7 L=0

Ötlet

- Az erdeti L listát elemenként „lebontjuk”.
- A mindenkori első elemet vagy L1, vagy L2 végére kell fűzni, ehhez két pointert fogunk használni: p1 az L1, p2 az L2 lista utolsó elemére fog mutatni.
- Mivel L1 és L2 is kezdetben üres, így p1 és p2 is nulláról fog indulni.

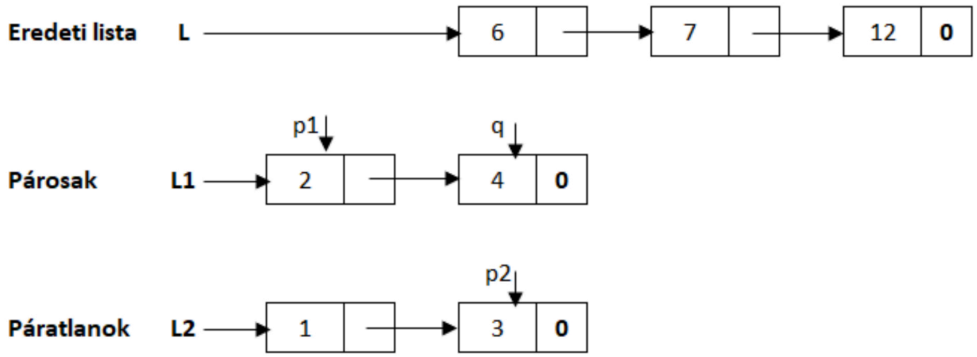
4-es kulcs feldolgozása

- q := L //q pointerbe beletesszük az átfűzendő elem címét,
- L := L → next //L a lista következő elemére mutasson. L-t eggyel arrébb állítjuk, L a 6-os kulcsú elemre fog mutatni.



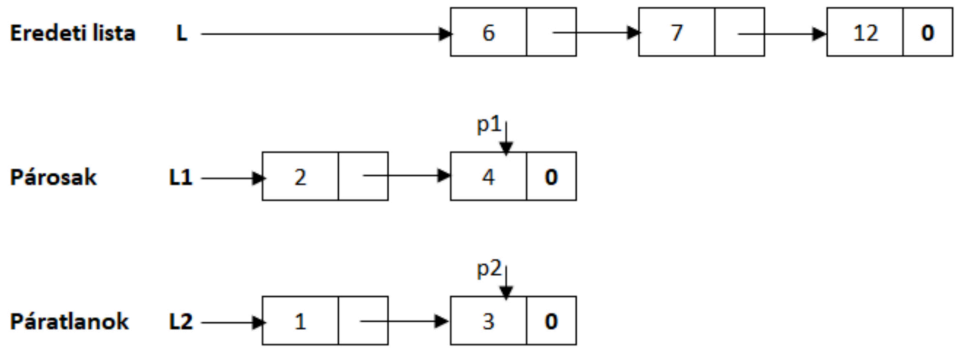
4-es kulcs befűzése L1 végére:

- p1 → next:=q //p1 utáni elem q lesz,
- q → next:=0 //mivel q című elem L1 utolsó eleme, pointerét nullára kell állítani

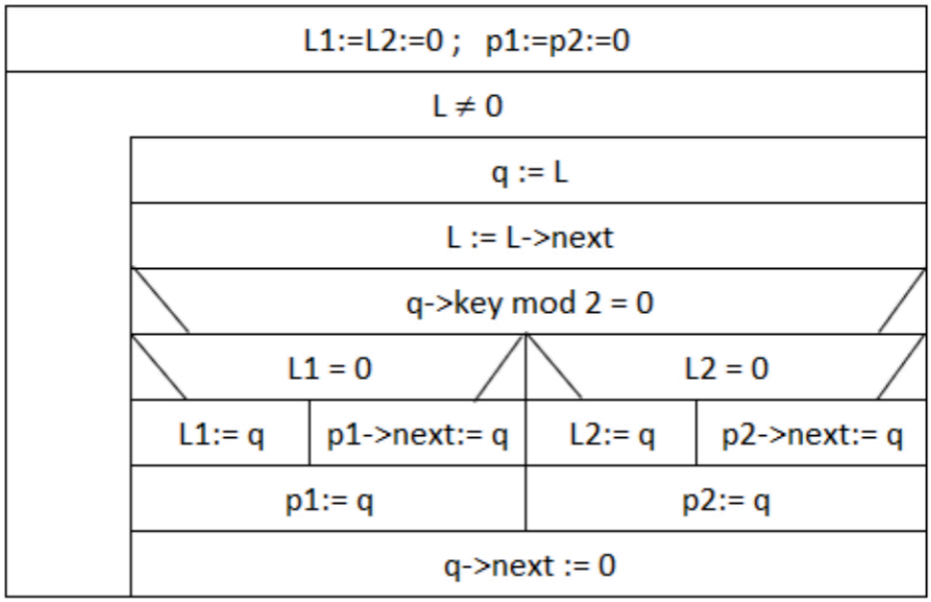


4-es kulcs befűzésének utolsó lépése:

- p1 :=q //p1-et módosítani kell, q lett az utolsó elem.

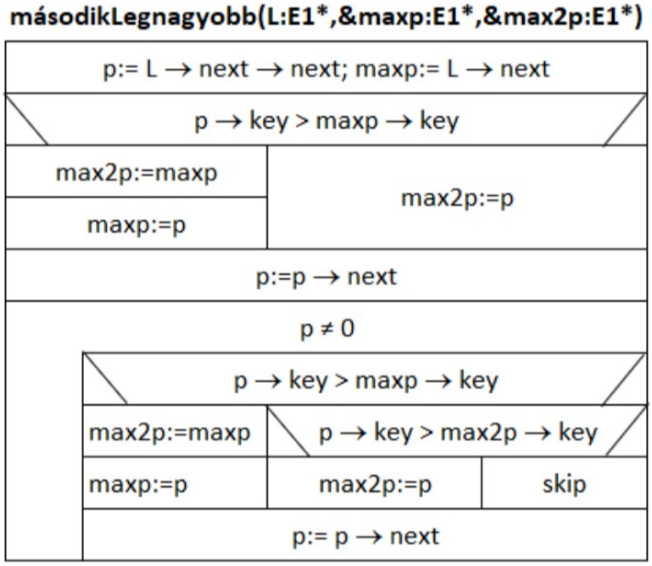


Szétfűz(&L:E1*,&L1:E1*,&L2:E2*)



H1L második legnagyobb

Egy rendezetlen egyirányú fejelemes listában (H1L) keressük meg a 2. legnagyobb elemet, egyszeri bejárással!
Feltehető, hogy a listánk legalább két eleme van (a fejelemen kívül).



Fontos az előfeltétel: a listának legalább két eleme van!

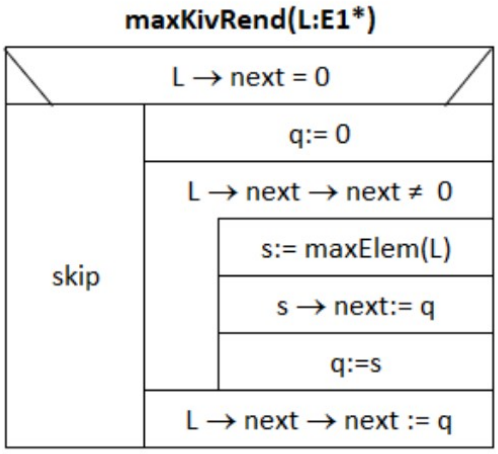
p a fejelem utáni második elemre mutat! (Ha esetleg üres a lista, ez elszállna!)

Elindulás az első két elem alapján:

maxp a nagyobbik, max2p a másik elemre mutat.

H1L maximum kiválasztásos rendezés

Rendezzünk egy H1L listát maximum kiválasztó rendezéssel. (Ügyeljünk, hogy a megoldó algoritmus üres listára is működjön!)



Üres listát külön kezeljük a ciklusfeltétel miatt.

q mutat a rendezett elemekből álló egyszerű lista első elemére.

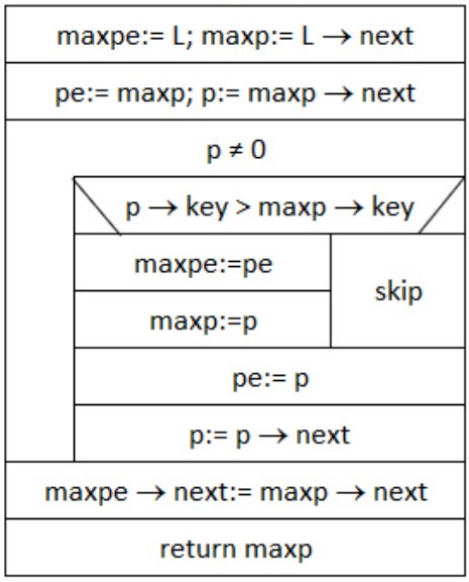
amíg legalább két elemet tartalmaz a lista...

megkeressük, és kifűzzük a lista legnagyobb kulcsú elemét

q című egyszerű lista elejére fűzzük a kapott elemet

Amikor L már csak egy elemet tartlmaz, a segéd listát befűzzük az első elem után.

A legnagyobb elemet megkereső, és kifűző algoritmus:



Megkeresi és kűzi a lista egyik legnagyobb elemét.

Előfeltétel, hogy legalább egy eleme legyen a listának.

maxp tartja nyilván a legnagyobb elemet, maxpe az előtte lévőt.

pe, és p pointerekkel járjuk be a listát.

Menet közben karban tartjuk maxpe és maxp pointereinket:

ha nagyobb kulcsot találunk, mint az eddigi legnagyobb,

maxpe-t pe-re és maxp-t p-re állítjuk.

Kifűzzük a maxp című elemet,

és visszatérünk a címével.

H1L rendezése beszúró rendezéssel

BeszúróRendezés(Fej)

```
p:=Fej->next
Fej->next:=NULL
p ≠ NULL
    q:=p
    p:=p->next
    Beszúr(Fej,q)
```

p pointer a lista első elemére mutat, ha üres lenne a lista, *p* =NULL lesz.

A Fejelem pointerét NULL-ra állítva, létrehozzuk az üres rendezett listát.

p pointer a beszúrandó elemre mutat, amíg NULL nem lesz a ciklusba belépve elvégezzük a beszúrást. Mivel *p* elem beszúrása során *p* mutatója el fog "romlani", így a beszúrás előtt *q* pointerben megjegyezzük a beszúrandó elem címét, és *p* pointerrel tovább lépünk a következő beszúrandó elem címére.

Beszúr függvény végzi a q paraméterben kapott elem befűzését a rendezettség szerinti helyére:

Beszúr(Fej,q)

```
s:=Fej
s->next ≠ NULL és
s->next->tart < q->tart
    s:=s->next
q->next:=s->next
s->next:=q
```

s pointert a Fejelemről indítjuk.
addig lépünk a listán az *s* utáni elemre, amíg van *s* után elem, és annak tartalma kisebb, mint a beszúrandó elem.

s arra az elemre mutat, amely *után* kell a listába beláncolni a beszúrandó elemet, melynek címe *q* pointerben van.

H1L szigorú monoton növekvő listává alakítása

L egy egyirányú, fejelemes listára mutat. A lista elemei egész számok, rendezetlen sorrendben, esetleg lehet üres is. Készítsen algoritmust, mely egyszer végigmegy a lista elemein, és törli azokat az elemeket, melyek nem nagyobbak, mint a listában előttük szereplő elem. (Így az eredmény egy szigorúan monoton növekvő lista lesz.)

C2L - hamazok uniója

<table>
<tr><th colspan="4">Unio(L1: E2*, L2: E2*)</th></tr>
<tr><td colspan="4">p := L1->next</td></tr>
<tr><td colspan="4">q := L2->next</td></tr>
<tr><td colspan="4">p ≠ L1 ∧ q ≠ L2</td></tr>
<tr><td></td><td>p->key < q->key</td><td>p->key = q->key</td><td>p->key > q->key</td></tr>
<tr><td></td><td rowspan="6">p := p->next</td><td>r := q->next</td><td>r := q->next</td></tr>
<tr><td></td><td>out(q)</td><td>out(q)</td></tr>
<tr><td></td><td>delete q</td><td rowspan="3">precede(q, p)</td></tr>
<tr><td></td><td>q := r</td></tr>
<tr><td></td><td rowspan="2">p := p->next</td></tr>
<tr><td></td><td>q := r</td></tr>
<tr><td colspan="4">q ≠ L2</td></tr>
<tr><td colspan="3">L1->prev->next := L2->next</td><td rowspan="5">skip</td></tr>
<tr><td colspan="3">L2->next->prev := L1->prev</td></tr>
<tr><td colspan="3">L2->prev->next := L1</td></tr>
<tr><td colspan="3">L1->prev := L2->prev</td></tr>
<tr><td colspan="3">L2->next := L2->prev := L2</td></tr>
</table>

C2L - halmazok metszete

<table>
<tr><th colspan="4">Intersect(L1: E2*, L2: E2*): E2*</th></tr>
<tr><td colspan="4">p := L1->next</td></tr>
<tr><td colspan="4">q := L2->next</td></tr>
<tr><td colspan="4">L3 := new E2</td></tr>
<tr><td></td><td colspan="3">p ≠ L1 ∧ q ≠ L2</td></tr>
<tr><td></td><td>p->key < q->key</td><td>p->key = q->key</td><td>p->key > q->key</td></tr>
<tr><td></td><td rowspan="5">p := p->next</td><td>r := new E2</td><td rowspan="5">q := q->next()</td></tr>
<tr><td></td><td>r->key := p->key</td></tr>
<tr><td></td><td>follow(L3->prev, r)</td></tr>
<tr><td></td><td>p := p->next</td></tr>
<tr><td></td><td>q := q->next</td></tr>
<tr><td colspan="4">return L3</td></tr>
</table>

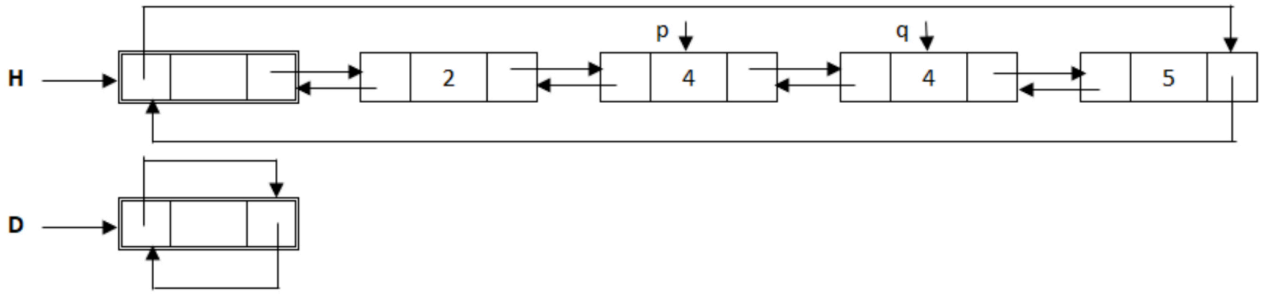
C2L bejárása és módosítása egy feladaton keresztül

A H pointer egy monoton növekvően rendezett, nem üres C2L fejelemére mutat. A D pointer egy üres C2L fejelemére mutat.

Készítsük el a duplDel(H,D:E2*) eljárást, ami a duplikált adatoknak csak az első előfordulását hagyja meg! A H lista tehát szigorúan monoton növekvő lesz. A feleslegessé vált elemekből hozzuk létre a D pointerű monoton növekvő C2L-t.

H lista egyszer járható be, az átfűzés konstans idejű legyen.

Ötlet



• H listát a p pointerrel fogjuk bejárni.

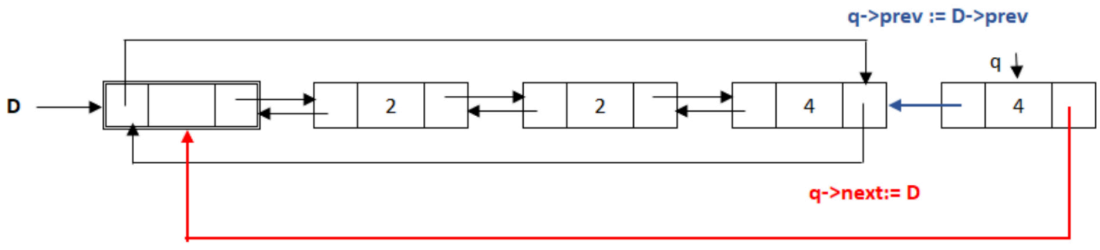
• Ha a p elem kulcsa megegyezik a p utáni elemnek a kulcsával, q pointert ráállítjuk a p utáni elemre, q elemet kifűzzük, majd átfűzzük D listába.

• Hogy D lista is növekvő legyen, mindig a lista végére, azaz a „fejelem elé” kell befűzni a q elemet.

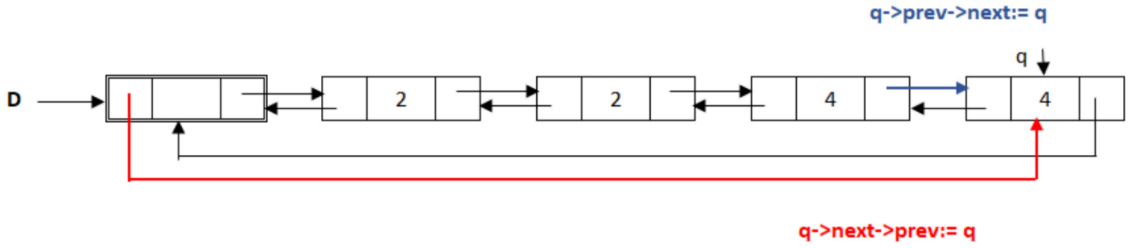
A befűző lépés szemléltetése

q című elem befűzése D pointerű C2L végére

1. q két pointerét ráállítjuk a két szomszédjára



2. a két szomszéd megfelelő pointereit q-ra kell állítani



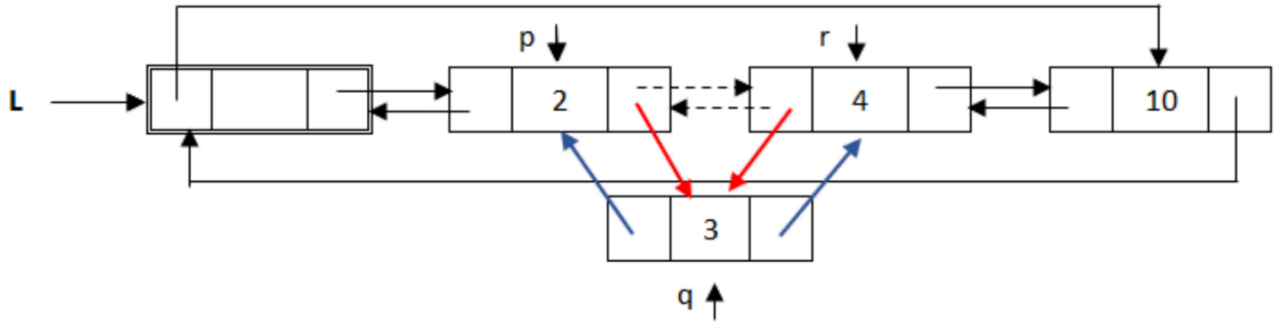
2022.03.08.

C2L műveletei

Precede

- egy listaelem beszúrása egy másik listaelem elé

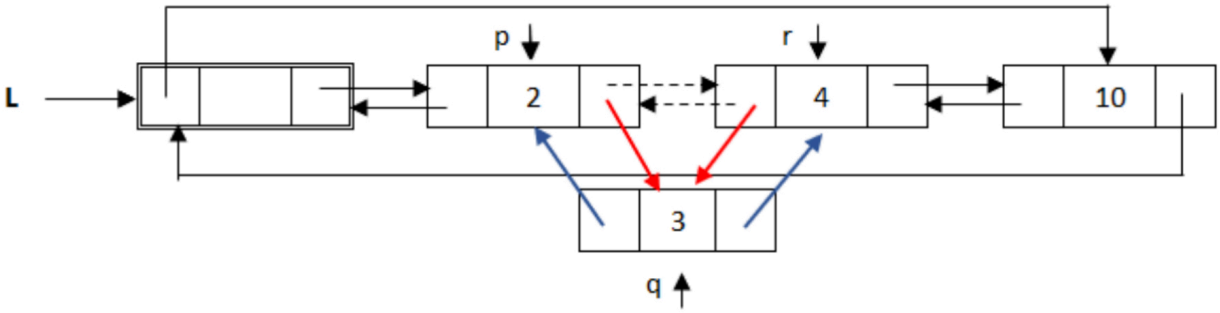
precede(q, r : E2*)
// insert $(*q)$ before $(*r)$
$p := r \rightarrow prev$
$q \rightarrow prev := p$; $q \rightarrow next := r$
$p \rightarrow next := r \rightarrow prev := q$



Follow

- az első paraméterben kapott listaelem után beszúrjuk a másikat

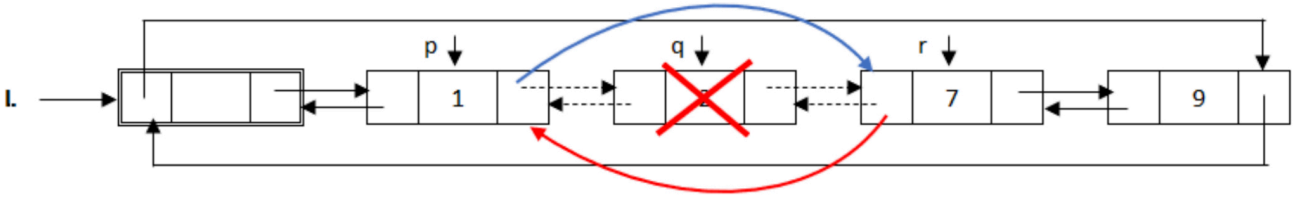
follow(p, q : E2*)
// after $(*p)$ insert $(*q)$
$r := p \rightarrow next$
$q \rightarrow prev := p$; $q \rightarrow next := r$
$p \rightarrow next := r \rightarrow prev := q$



Unlink

- a megadott listaelem kifűzése

unlink(q : E2*)
// remove $(*q)$
$p := q \rightarrow prev$; $r := q \rightarrow next$
$p \rightarrow next := r$; $r \rightarrow prev := p$
$q \rightarrow prev := q \rightarrow next := q$



Az algoritmus

duplDel(H,D:E2*)

D->next:=D->prev:=D	
p:= H->next	
p->next ≠ H	
p->key = p->next->key	
q:=p->next	p:=p->next
q->prev->next:=q->next	
q->next->prev:=q->prev	
q->next:= D	
q->prev:= D->prev	
q->prev->next:= q	
q->next->prev:= q	

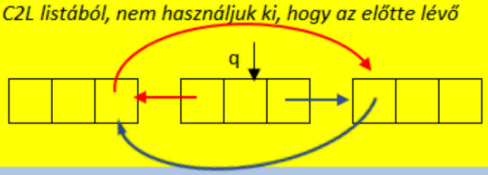
D fejelem pointereit önmagára állítjuk (feladat szövege alapján elhagyható lenne)

P-vel a H első elemére állunk.

Mivel a p utáni elemet fogjuk a p elemmel összehasonlítani, itt azt vizsgáljuk, hogy p után még nem a fejelem következik.

Ha a p elem és a p utáni elem kulcsa azonos, akkor q-t a p utáni elemre állítjuk.

A q című elem kifűzése C2L listából, nem használjuk ki, hogy az előtte lévő elemet ismerjük:



A q című elem befűzése a fejelem elé, elsőként q pointereit állítjuk be: q után a fejelem jön, az utolsó elem (D->prev) lesz a q előzője.

Majd q két szomszédjának pointereit állítjuk be: a q előtti elem rákövetkezője q, továbbá a q utáni elemnek az előzője szintén q. FONTOS, hogy előtte q->prev és q->next már be legyen állítva!

Prímtényezős felbontásból lkkt

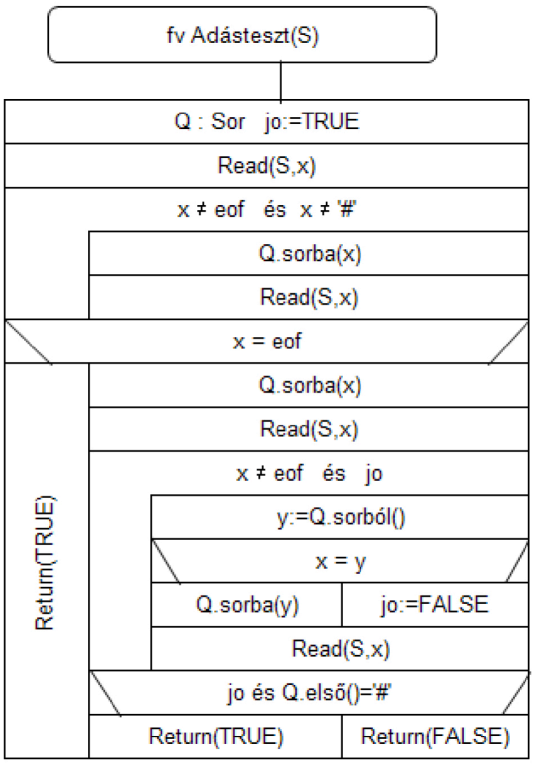
LKKT(Q1: Queue, Q2: Queue): Queue

Q3: Queue			
Q2.add(-1)			
¬ Q1.isEmpty() ∨ Q2.first() ≠ -1			
	Q2.first() = -1 ∨ (¬ Q1.isEmpty() ∧ Q1.first() < Q2.first())	¬ Q1.isEmpty() ∧ Q2.first() ≠ -1 ∧ Q1.first() = Q2.first()	Q1.isEmpty() ∨ (Q2.first() ≠ -1 ∧ Q1.first() > Q2.first())
	Q3.add(Q1.rem())	Q3.add(Q1.rem())	Q3.add(Q2.first()
		Q2.add(Q2.rem())	Q2.add(Q2.rem())
Q2.rem() //-1 végjel eltüntetése			
return Q3			

Adásteszt

Egy rádióadónak az a feladata, hogy újra és újra leadja ugyanazt a jelsorozatot. A jelsorozat végén egy szünet-jelet ad, a példákban ezt # jellel jelöltük. Az adást egy szekvenciális fájlban rögzítettük, készítsünk algoritmust, mely sor segítségével eldönti, hogy volt-e hiba az adás során. Az input fájlt csak egyszer olvashatjuk végig.

Példák:
a a 0 b # a a 0 b # a a 0 b - hibátlan
c 0 b # c 0 b # 0 b - hibás
a a a b c - hibátlan (nem volt ismétlés)
a b c d # a b c d # a b c d a - hibás



Beolvassuk a minta jelsorozatot, és berakjuk egy Q sorba.

Ha elértük a fájl végét, csak egy adás volt, és az helyes.
A '#' jelet is berakjuk a sorba.
Addig olvasunk, és hasonlítunk (az inpuról jövő jelet a sor elejéről vett jellel), amíg vagy elfogy a sorozat, vagy eltérést találunk.
A sorba mindig visszatesszük a kivett jelet!

Ha elértük a fájl végét, és a sorban épp egy '#' jel következik, akkor helyes volt az adás!